

Agenda

- Adding structure to Python code
- Functions
- Namespaces
- Modules

Code Structure

Adding structure to Python code

- There are many good reasons why we want to organize our code and add structure to it
 - Reuse: We don't want to do things twice
 - Readability: structured code can be read like a language
 - Maintainability: If we don't have a good structure in code, we can spend a lot time with searching
- In Python we have different concepts to give structure to our code:
 - Functions
 - Namespaces
 - Modules
 - Classes (we will come back to this great concept at the end of this course)

Functions

Usage of Functions

- Functions allow us to define a block of code that can be called from anywhere
- Functions accept *parameters* (arguments) as input into the code block
- Optionally, functions can *return* a value
- As soon as you repeat the same or similar sequence of code, consider writing a function
- Functions are called with the function name, followed by ()
 - If the function accepts parameters, they are listed inside (), separated with comma
 - If the function returns a value, it can be assigned to a variable

Here is an example:

Usage of Functions

- Functions allow us to define a block of code that can be called from anywhere
- Functions accept *parameters* (arguments) as input into the code block
- Optionally, functions can *return* a value
- As soon as you repeat the same or similar sequence of code, consider writing a function
- Functions are called with the function name, followed by ()
 - If the function accepts parameters, they are listed inside (), separated with comma
 - If the function returns a value, it can be assigned to a variable

Here is an example:

```
In [1]: my_number = max(3, 4, 2) # calling the function with 3 arguments and as  
        print(my_number)        # also print is a function
```

Definition of Functions

- To define a function, we use the `def` keyword, followed by the function name
- After the function name we declare in () which parameters are allowed
- If we want to return a value, we can do this anywhere with the `return` statement, followed by the return value(s)

Definition of Functions

- To define a function, we use the `def` keyword, followed by the function name
- After the function name we declare in () which parameters are allowed
- If we want to return a value, we can do this anywhere with the `return` statement, followed by the return value(s)

```
In [2]: # Define a function that takes two arguments and returns the bigger one  
def bigger_of_two(a, b):  
    if a > b:  
        return a  
    else:  
        return b  
  
# Call the function with two arguments and print the result  
print(bigger_of_two(4, 3))
```

4

Scope of Variables

- Variables that are first used in a function are only known inside this function
- However, functions can access variables of the outer scope (e.g. global variables)
- Variables in the outermost scope are called "global variables"
- Variables in a closed inner scope are called "local variables"

Scope of Variables

- Variables that are first used in a function are only known inside this function
- However, functions can access variables of the outer scope (e.g. global variables)
- Variables in the outermost scope are called "global variables"
- Variables in a closed inner scope are called "local variables"

```
In [4]: x = 8
def some_function():
    print("Value from outer scope:", x) # x is accessible from the inner scope
    y = 3
    print("Value from same scope:", y) # y is not accessible from the outer scope

some_function()
print("Value from inner scope:", y)
```

```
Value from outer scope: 8
Value from same scope: 3
```

```
-----
-----
NameError                                Traceback (most recent
call last)
Cell In[4], line 8
      5     print("Value from same scope:", y) # y is not access
ible from the outer scope
      7 some_function()
----> 8 print("Value from inner scope:", y)

NameError: name 'y' is not defined
```

Assigning multiple values

- In Python we can assign multiple values to multiple variables in one line
- Accordingly, we can write functions that return multiple values, which can be very handy

Assigning multiple values

- In Python we can assign multiple values to multiple variables in one line
- Accordingly, we can write functions that return multiple values, which can be very handy

```
In [ ]: def get_min_max(list):  
        minimum = min(list)  
        maximum = max(list)  
        return minimum, maximum # returning multiple values  
  
list_min, list_max = get_min_max([1, 2, 3, 4, 5]) # multiple assignment  
print(list_min)
```

More About Functions in Python

- Default Parameter Value: You can define default values in case the function is called without parameters

More About Functions in Python

- Default Parameter Value: You can define default values in case the function is called without parameters

```
In [ ]: def default_argument(a, b, c=3): # default value for c
        return a + b + c
        print(default_argument(1, 2)) # c will be 3
```

More About Functions in Python

- Default Parameter Value: You can define default values in case the function is called without parameters

```
In [ ]: def default_argument(a, b, c=3): # default value for c
        return a + b + c
print(default_argument(1, 2)) # c will be 3
```

- Arbitrary Arguments, args: If you do not know how many arguments will be passed into your function, add a before the argument name

More About Functions in Python

- Default Parameter Value: You can define default values in case the function is called without parameters

```
In [ ]: def default_argument(a, b, c=3): # default value for c
        return a + b + c
print(default_argument(1, 2)) # c will be 3
```

- Arbitrary Arguments, args: If you do not know how many arguments will be passed into your function, add a before the argument name

```
In [ ]: def arbitrary_argument(*args): # variable number of arguments
        sum = 0
        for a in args:
            sum += a
        return sum

print(arbitrary_argument(1, 2, 4, 6))
```

More About Functions in Python

- Default Parameter Value: You can define default values in case the function is called without parameters

```
In [ ]: def default_argument(a, b, c=3): # default value for c
        return a + b + c
print(default_argument(1, 2)) # c will be 3
```

- Arbitrary Arguments, *args*: If you do not know how many arguments will be passed into your function, add *a* before the argument name

```
In [ ]: def arbitrary_argument(*args): # variable number of arguments
        sum = 0
        for a in args:
            sum += a
        return sum

print(arbitrary_argument(1, 2, 4, 6))
```

- There is more we can do with functions, but this is a good start.

Modules

Using Modules I

- One of the biggest advantages of Python is the incredible number of modules that are available
- Modules are pieces of code that exist somewhere else (typically in another file)
- Modules can be imported with the `import module_name` statement
- The module name can be any installed Python module,
- or the name of a file somewhere in your project (name without the '.py' ending)
- Functions in a module are accessible using the module name and the `.` operator

Using Modules I

- One of the biggest advantages of Python is the incredible number of modules that are available
- Modules are pieces of code that exist somewhere else (typically in another file)
- Modules can be imported with the `import module_name` statement
- The module name can be any installed Python module,
- or the name of a file somewhere in your project (name without the '.py' ending)
- Functions in a module are accessible using the module name and the `.` operator

```
In [ ]: import math # importing a module

math.sqrt(16) # using a function from the module
```

Using Modules II

- A module can be organized into submodules by placing them in a folder or subfolder within your project
- The module name will then be `module.submodule` (according to the folder structure)
- To access a function, you will have to write something like `module.submodule.function()`
- In this case, we want to create a shortcut.
- In Python, this can be done using the `as` keyword: `import folder.module as mod`

Using Modules II

- A module can be organized into submodules by placing them in a folder or subfolder within your project
- The module name will then be `module.submodule` (according to the folder structure)
- To access a function, you will have to write something like `module.submodule.function()`
- In this case, we want to create a shortcut.
- In Python, this can be done using the `as` keyword: `import folder.module as mod`

```
In [ ]: import pandas.plotting as pdp # importing a submodule
pdp.lag_plot() # using a function from the submodule
```

Installing Modules

- As mentioned earlier, Python has a vast variety of modules (currently about 570k registered packages)
- You can find the official registry here: <https://pypi.org/>
- Fortunately we don't have to search and copy them from the internet
- Python has a package management tool, called pip that helps us to install modules

```
pip install <module name>  
pip install pandas
```

if you have several Python versions you may have to use the specific pip command

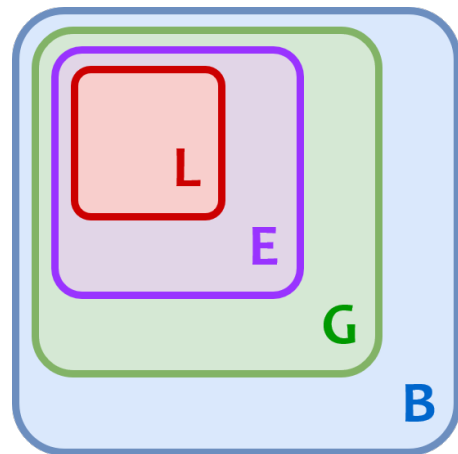
```
pip3 install <module name>
```

Crating a Module

- At some point your code will get bigger
- And also you might want to reuse your own code in other projects
- In this case it is a good idea to start organizing your code in modules
- Creating a modules is as easy as creating a new .py file
- Let's code a simple example.

Namespaces

Namespace Rules



- Now that we know functions and modules, it's time to have a closer look at namespaces
- A namespace is the scope in which a name (variable, function, class, ...) is valid
- Names must be unique in their namespaces
- Python recognizes 3 types of namespaces with the following rules:
 1. Local: If you refer to x inside a function, then the interpreter first searches for it in the innermost scope that's local to that function.
 2. Enclosing: If x isn't in the local scope but appears in a function that resides inside another function, then the interpreter searches in the enclosing function's scope.
 3. Global: If neither of the above searches is fruitful, then the interpreter looks in the global scope next.
 4. Built-in: If it can't find x anywhere else, then the interpreter tries the built-in scope.

Working with Namespaces

- In Python, functions, modules (and classes) act as enclosing and local namespaces
- You might have noticed: modules automatically act as a namespace
- Their structure determines the scope of names
- In terms of memory:
 - Global variables remain in RAM at all times
 - Local variables only use memory while the object defining the namespace exists

Working with Namespaces

- In Python, functions, modules (and classes) act as enclosing and local namespaces
- You might have noticed: modules automatically act as a namespace
- Their structure determines the scope of names
- In terms of memory:
 - Global variables remain in RAM at all times
 - Local variables only use memory while the object defining the namespace exists

```
In [8]: x = 1
def namespaces():
    def subspaces():
        z = 3
        print(pow(3,2)) # access namespaces from the built-in scope
        print(x) # access x from the global scope
        print(y) # access y from the enclosing scope
        print(z) # access z from the local scope

    y = 2
    subspaces()
    print(y) # access y from the local scope
    print(z) # z is not accessible from the outer scope (NameError)

namespaces()
```

```
9
1
2
3
```


For the Geeks

- Careful about redeclaration!
- While an enclosing variable `x` can be read in an inner namespace, a new variable will be created as soon as you assign a value to it

For the Geeks

- Careful about redeclaration!
- While an enclosing variable `x` can be read in an inner namespace, a new variable will be created as soon as you assign a value to it

```
In [5]: def namespaces():  
        def subspaces():  
            x = 2 # x is now a local variable  
            print(x)  
  
            x = 1 # x is a local variable of namespaces  
            subspaces()  
            print(x)  
  
namespaces()
```

2
1